Backlink Admissibility in Cyclic Proofs for the Calculus of Inductive Constructions

Gavin Mendel-Gleason

Scidonia Limited
gavin@scidonia.ai

Abstract. Cyclic proofs close goals by backlinking to previously-seen configurations, discovering induction principles post-hoc rather than prescribing them upfront. We prove that backlinks are *admissible* in the Calculus of Inductive Constructions (CIC): every cyclic proof can be transformed into a standard CIC proof that uses only the native fixpoint binder (`tFix`) for recursion. The transformation is mechanised in Rocq with 0 axioms. The proof is a corollary of a contextual-equivalence theorem for a supercompiler: the residual program read back from a valid cyclic proof graph is CIU-equivalent to the source and contains no backlink constructs.

1 Introduction

Cyclic proof theory [1,2] extends standard proof systems with backlink rules that allow a premise to reference a previous goal, forming a directed cycle. The cycle is sound if it satisfies a global *trace condition*: every cycle must contain a *progress event*—a case-split on a structurally smaller term. Brotherston and Simpson prove that for first-order logic with inductive definitions, backlinks are *admissible*: every cyclic proof corresponds to a standard proof with an explicit induction rule.

This paper extends their result to the Calculus of Inductive Constructions (CIC) [3,4], the type theory underlying Rocq. The central theorem is:

Theorem 1 (Backlink admissibility). *For every valid cyclic proof (rooted pre-proof plus well-founded ranking) in our calculs, there exists a standard CIC term using only `tFix` for recursion that is contextually equivalent to the root goal.*

The proof is a corollary of the CIU soundness theorem mechanised in Rocq. The cyclic proof graph b is read back into a residual CIC term via a *residualisation* function that replaces backlinks with `tFix` binders. The CIU theorem proves the residual is observationally indistinguishable from the source. Hence the residual *is* the standard proof.

2 Calculus

We work with a core CIC: terms are variables, sorts, dependent products, λ-abstractions, applications, fixpoints, inductive type formers, constructors (`tRoll`), and dependent case expressions (`tCase`). Typing follows standard CIC rules.

A *configuration* is a typing judgement $\Gamma \vdash t : A$. A *configuration graph* is a finite directed graph where each vertex v carries a configuration $\mathsf{label}(v)$ and a list of successors $\mathsf{succs}(v)$. It is a *rooted pre-proof* if every vertex satisfies a local sequent rule: $\mathsf{label}(v)$ is the conclusion of a rule whose premises are the labels of its successors.

3 Cyclic proofs

A *cyclic backlink* closes a goal by pointing to a previously-seen vertex whose configuration is an instance of the current goal under an explicit substitution. A pre-proof becomes a *cyclic proof* when it satisfies a global progress condition: every directed cycle must contain at least one *progress edge*—a case-split on a neutral scrutinee. This guarantees that the recursion is well-founded.

The progress condition is enforced by a *budget trace* construction [1]: vertices are lifted to pairs (v, k) where $k \in [0, |V|]$. Progress edges consume one unit of budget ($k \mapsto k - 1$); non-progress edges preserve it ($k \mapsto k$). Since every cycle contains a progress edge, the lifted graph is acyclic.

4 Residualisation eliminates backlinks

The backlink rule appears only in the cyclic proof graph. When we *residualise*—read back the graph into a CIC term—backlinks are replaced by `tFix` binders. Specifically:

- A vertex with no successors (a leaf) residualises to its own term.
- A vertex with one successor residualises by applying the driving step (unfold, β-reduce, apply constructor, etc.) to the successor's residual.
- A vertex that is the target of a backlink residualises to a `tFix` whose body is the residual of the vertex's successors, and whose recursive call sites reference the `tFix` binder.

The result is a well-typed CIC term that uses only standard constructors (`tLam`, `tApp`, `tFix`, `tCase`, `tRoll`). No backlink constructs remain.

5 CIU equivalence

The soundness guarantee is *CIU equivalence* (contextual equivalence under all closing substitutions):

$$t \approx_{\text{ciu}} u \;\equiv\; \forall \sigma, v.\, (t[\sigma] \Downarrow v \iff u[\sigma] \Downarrow v)$$

The mechanised theorem (`supercompile_ciu_soundness_untyped`) establishes:

Theorem 2 (CIU soundness). *For any environment Σ, fuels f_s, f_r, context Γ, term t, and type A, if the cyclic proof construction succeeds with the trace condition,*

$$SC.\mathsf{supercompile\_jTy\_tc}\ f_s\ \Sigma\ \Gamma\ t\ A = \mathsf{Some}\,(v, b)$$

then the residualised term is CIU-equivalent to the source:

$$t \approx_{\mathrm{ciu}} SC.\mathsf{residualise\_cfg}\ f_r\ \Sigma\ b\ v\ 0\ \emptyset.$$

The proof (mechanised in Rocq, 0 axioms) is by fuel induction on the residualiser, with the trace condition excluding non-terminating self-loops. Congruence lemmas lift CIU through all nine term constructors.

6 Admissibility

Let $\mathcal{C}(b, v)$ denote that b is a rooted, ranked cyclic proof with root v (local rule satisfaction plus the budget-trace ranking of Section 3).

Theorem 3 (Backlink admissibility).

$$\forall\, b, v.\ \mathcal{C}(b, v) \implies \exists\, t_{\mathsf{std}},\ \mathsf{label}(b, v) = \Gamma \vdash t : A\ \wedge\ t \approx_{\mathrm{ciu}} t_{\mathsf{std}}.$$

In words: every rooted cyclic proof in the sequent calculus can be read back to a standard CIC term (using only `tFix` for recursion) that is observationally indistinguishable from the root goal.

Proof. Take $t_{\mathsf{std}} = \mathrm{SC}.\mathsf{residualise\_cfg}\ f_r\ \Sigma\ b\ v\ 0\ \emptyset$. The residual consists only of standard CIC constructors by construction (Section **??**). CIU equivalence follows from Theorem 2. □

The mechanised proof (`BacklinkAdmissible.v`) is a four-line corollary: extract the residual via `supercompile_jTy_tc`, apply `supercompile_ciu_soundness_untyped`, reflexivity.

7 Discussion

The key observation is that the admissibility transformation is not a separate construction—it is the same residualisation that the supercompiler already performs. The CIU theorem proves equivalence; the residualiser eliminates backlinks; the result follows trivially.

Compared to Brotherston and Simpson's admissibility result for first-order logic, our proof is simpler because (a) the cyclic proof graph is finite and constructed by a deterministic process rather than an arbitrary proof search, and (b) the residualisation function explicitly replaces backlinks with `tFix`, making the transformation syntactic rather than semantic.

Acknowledgements. Thanks to James Brotherston for the question that motivated this paper.

References

1. Brotherston, J.: Cyclic proofs for first-order logic with inductive definitions. In: TABLEAUX (2006)
2. Brotherston, J., Simpson, A.: Sequent calculi for induction and infinite descent. In: LICS (2011)
3. Coquand, T., Huet, G.: The calculus of constructions. Information and Computation **76**(2–3), 95–120 (1988)
4. Paulin-Mohring, C.: Inductive definitions in the system Coq: Rules and properties. In: Typed Lambda Calculi and Applications. pp. 328–345. Springer (1993)